Configuration Management, Version Control with Git, and Change Management

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain software configuration management and why controlled change matters.

• Identify configuration items, baselines, versions, and change requests.

• Describe Git's working tree, staging area, local repository, and remote repository.

• Use common Git commands to record, inspect, branch, merge, and share changes.

• Apply a simple change-management workflow to a College Event Management System (CEMS).

Running case study: A student team is developing CEMS and needs to coordinate changes to event registration, capacity rules, and organizer access without losing working code.

1. What Is Software Configuration Management?

Software Configuration Management (SCM) is the disciplined process of identifying, organizing, tracking, and controlling changes to software artefacts throughout their lifecycle. These artefacts may include source code, requirements, UML diagrams, test cases, build scripts, configuration files, database scripts, and user documentation.

Without configuration management, team members may overwrite one another's work, use different versions of a requirement, release untested code, or be unable to reproduce a previous build. SCM makes change history and product versions traceable.

Example: The CEMS team changes the rule for event capacity. The team needs to know which requirement approved the change, which code and tests were modified, who reviewed it, and which release contains the change.

2. Key SCM Concepts

Concept	Meaning	CEMS example
Configuration item	An artefact selected for controlled management	SRS, registration service, test plan, deployment configuration
Version	A recorded state or revision of an artefact	Registration service before and after a capacity-rule fix
Baseline	An agreed reference set of artefacts used for further work or comparison	Approved SRS and design at the end of requirements review
Change request	A documented request to modify an approved artefact or baseline	Add a waitlist when an event reaches capacity
Configuration status accounting	Recording and reporting the state and history of items and changes	Which requests are approved, implemented, tested, and released?
Configuration audit	Checking that a release contains the required, approved items	Confirm that the release includes the correct code, configuration, and tests

3. A Controlled Change Workflow

1. Record the change request and explain the business need.

2. Analyze impact on requirements, design, code, tests, schedule, cost, security, and documentation.

3. Approve, reject, defer, or request clarification according to the team's authority and process.

4. Update the relevant work items and identify the affected configuration items.

5. Implement the change in a controlled branch or workflow.

6. Review the change and run appropriate automated and manual tests.

7. Merge and prepare a release using the agreed approval process.

8. Record the version, release notes, test evidence, and any follow-up work.

A small student project may use a lightweight process; a safety-critical or regulated project may require more formal approvals and records. The level of control should fit the risk and context.

4. What Is Version Control?

Version control records changes to files over time so a team can inspect history, compare revisions, collaborate, and restore earlier versions when appropriate.

Without version control	With version control
Files named final, final-new, final-latest, final-fixed	Meaningful commits and traceable history
Team members overwrite shared files	Changes can be developed and reviewed separately
Hard to identify when a bug appeared	History and comparisons help investigate changes
Difficult to recover a previous state	Earlier commits can be inspected or restored safely

Version control does not replace backups, code review, testing, access control, or release management. A bad change can still be committed; version control makes it easier to trace and manage that change.

5. Git: The Main Areas

Area	Purpose	Typical operation
Working tree	Files currently being edited	Edit a source file
Staging area / index	Selects the changes to include in the next commit	git add
Local repository	Stores commits and history on your machine	git commit
Remote repository	Shared repository hosted on a service or server	git push / git pull

> A commit is a recorded snapshot of selected changes, accompanied by metadata such as a message and author information. A branch is a movable reference to a line of development. A remote is a named reference to another repository, commonly a shared one.

6. Essential Git Commands

Command	Purpose	Example / note
git --version	Check whether Git is installed	Prints installed version
git init	Initialize a repository in the current directory	Use for a new local project
git clone URL	Copy an existing repository	Creates a local working copy
git status	Show changed, staged, and untracked files	Run frequently
git add file	Stage selected changes	git add src/registration.js
git add .	Stage changes under the current directory	Review status first
git commit -m "message"	Record staged changes	Use a concise, meaningful message
git log --oneline	Inspect recent commit history	Useful for tracing changes
git diff	View unstaged differences	Check what changed before staging

Command	Purpose	Example / note
git diff --staged	View staged differences	Review what will be committed
git branch	List local branches	Shows current branch marker
git switch -c feature/name	Create and switch to a branch	For isolated work
git merge branch-name	Merge a branch into the current branch	Resolve conflicts if they occur
git pull	Fetch and integrate remote changes	Review team workflow and possible conflicts
git push	Send local commits to a remote	Remote permissions and upstream may be needed

7. A Typical Git Workflow

Example: a developer adds a validation rule for CEMS registration.

```
git status
git switch -c feature/registration-validation
# Edit the relevant source file and tests
git diff
git add src/registration.js tests/registration.test.js
git diff --staged
git commit -m "Add registration validation"
git push -u origin feature/registration-validation
```

After pushing, the developer can open a pull request or merge request for review. The team reviews the change, checks test results, discusses issues, and merges according to its repository policy. Exact commands may differ depending on whether the repository already has a remote and whether the branch tracks an upstream branch.

8. Branching and Collaboration

Branches allow people to work on changes without immediately changing the main development line. Common patterns include short-lived feature branches and a protected main branch. The appropriate strategy depends on team size, release frequency, and tooling.

Practice	Why it helps	Caution
Small, focused commits	Makes history easier to review and understand	Avoid vague messages such as “changes”
Short-lived branches	Reduces long-running divergence	Integrate regularly and test before merging
Code review / pull request	Adds peer feedback and traceability	Review behavior, tests, security, and maintainability
Protected main branch	Can require checks before merging	Configure permissions and rules for the actual team
Meaningful .gitignore	Avoids tracking generated or local-only files	Do not ignore files needed to build or run the project

9. Merge Conflicts

A merge conflict occurs when Git cannot automatically combine changes, often because the same part of a file was edited differently. A conflict is not a sign that Git has lost all work; it signals that a person must decide the intended combined result.

1. Read the conflicted file and understand both changes.
2. Discuss the intended behavior with the relevant teammate if necessary.
3. Edit the file to the correct combined result and remove conflict markers.
4. Run relevant tests and inspect the final diff.

5. Stage the resolved file and complete the merge or commit according to the current workflow.

> Never resolve a conflict by blindly keeping one side without understanding the effect. Check business rules and tests, especially when the conflict involves authorization, data integrity, or registration capacity.

10. Tags, Releases, and Traceability

A tag can mark an important commit, such as a release candidate or version. A release should be traceable to its source revision, configuration, test evidence, and approved change requests.

```
git tag -a v1.0.0 -m "CEMS first release"
git show v1.0.0
git push origin v1.0.0
```

These commands illustrate annotated tags and sharing a tag; confirm the team's release naming policy before using them. A tag is a reference, not a substitute for a release checklist or backup.

11. Change Management and Git Are Related but Different

Configuration / change management	Git
Defines how changes are requested, evaluated, approved, tracked, and released	Records source-file changes and commit history
Can cover requirements, diagrams, test plans, code, infrastructure, and documents	Primarily manages files and revisions within repositories
Includes impact analysis and approval rules	Provides branches, diffs, commits, merges, and tags
Establishes governance and traceability expectations	Supports implementation of that process for version-controlled artefacts

> Git is a tool that supports configuration management. It is not the entire configuration-management process.

12. Classroom Practical: Git for CEMS

Work in pairs or small teams. Use a practice repository; do not experiment on a production repository.

1. Create a project folder and initialize Git, or clone a prepared repository.
2. Add a README describing CEMS and commit it with a meaningful message.
3. Create a feature branch named feature/event-list.
4. Add a small change and a test file, inspect the diff, stage selected files, and commit.
5. Push the branch to a shared remote if one is available.
6. Open a pull request or demonstrate a peer-review checklist.
7. Simulate or discuss a merge conflict and document how it was resolved.
8. Tag a practice release and explain which commit it identifies.

> **Deliverable:** repository URL or local evidence, commit history, branch/merge demonstration, README, and a short change record linking requirement → code change → tests.

13. Common Mistakes

• Committing passwords, API keys, private certificates, or sensitive personal data.

• Using one large commit for unrelated changes.

• Committing generated build folders or local environment files unnecessarily.

• Pulling or merging without understanding the team's workflow or resolving conflicts carefully.

• Treating a successful commit as proof that code is correct.

• Changing an approved requirement without recording impact and approval.

• Tagging a release without checking the exact commit and required tests.

14. Review Questions with Suggested Answers

1. What is software configuration management?

Suggested answer: Identifying, tracking, and controlling software artefacts and their changes across the lifecycle.

2. What is a baseline?

Suggested answer: An agreed reference set of artefacts used for comparison and controlled change.

3. How does a risk differ from a change request?

Suggested answer: A risk is an uncertain event that may affect objectives; a change request proposes modifying an artefact or baseline.

4. What is the Git staging area?

Suggested answer: The area where selected changes are prepared for the next commit.

5. What does git status show?

Suggested answer: The state of tracked and untracked files, including staged and unstaged changes.

6. Why use a branch?

Suggested answer: To isolate work so it can be developed and reviewed before integration.

7. What is a merge conflict?

Suggested answer: A situation where Git cannot automatically combine changes and a person must resolve the intended result.

8. Does Git replace testing and backups?

Suggested answer: No. It supports history and collaboration but does not replace quality checks or backup practices.

9. What should a change-impact analysis consider?

Suggested answer: Requirements, design, code, tests, schedule, cost, security, documentation, and dependencies.

10. How are Git and change management related?

Suggested answer: Git records file revisions; change management governs how changes are requested, assessed, approved, tested, and released.

15. Glossary

Term	Meaning
SCM	Software Configuration Management
Configuration item	Artefact selected for controlled management
Baseline	Approved reference state for comparison and change
Commit	Recorded set of staged changes in Git history
Branch	Movable reference to a line of development
Remote	Named reference to another repository
Merge conflict	Changes Git cannot combine automatically
Pull request	Proposal to review and integrate changes into another branch
Tag	Named reference often used to identify a release or milestone
Change-impact analysis	Assessment of effects a proposed change may have

16. Homework

1. Create a Git repository for a small Library Management System and write a clear README.
2. Make at least three meaningful commits and explain what each commit records.
3. Create a feature branch, make a change, and merge it after reviewing the diff.

4. Write a change request for adding a waitlist to CEMS. Include rationale, impact areas, acceptance criteria, and approval status.

5. Explain how you would trace a production release back to its source commit, change request, and test evidence.

Next chapter: Agile Project Management, Scrum Roles and Events, Product Backlog, and Sprint Planning.